

Кандидатогенерация и метрики

ШАД, Рекомендательные системы
2025/26, лекция 2

Даня Ткаченко

Руководитель службы ML-сервисов,
Яндекс Лавка

Даня Ткаченко

Head of ML в Яндекс Лавке

- Окончил Мехмат МГУ
- Почти 8 лет занимаюсь рекомендательными системами в Яндексе
 - Прошел путь от младшего разработчика до руководителя службы
 - Строил рекомендательные рантаймы от самых больших до самых маленьких
- Ex. Яндекс Поиск, Дзен



План занятия

На лекции:

1. Флоу работы типичной рексистемы
2. Кандидатогенерация и ее метрики качества
3. Виды и примеры кандидатогенераторов
4. Организационное про контекст и домашку

На семинаре:

1. Напишем бейзлайн для контекста
2. Попробуем его засабмитить
3. Расскажем про домашку

Флоу работы типичной рексистемы

Часть 1



Типичная рекомендательная система



1. Мы хотим сделать персональные рекомендации
2. Мысль с прошлой лекции: научимся считать число, которое будет показывать, насколько хорошо конкретный айтем подходит конкретному пользователю
3. Тогда все просто: для фиксированного пользователя посчитаем это число для всех айтемов, посортируем по нему, покажем пользователю то, что получится

Проблема

1. Проблема: представим, что мы условный YouTube
2. На прошлой лекции мы выяснили, что там 20В видео
3. Обычно в рантайме в зависимости от продукта у нас есть от 100 до 500 мс на то, чтобы успеть отдать ответ пользователю
4. Если мы хотим успевать ранжировать прямо в онлайн, когда на сервер прилетает запрос, нам будет:



Вариант 1:

Оффлайн-рекомендации

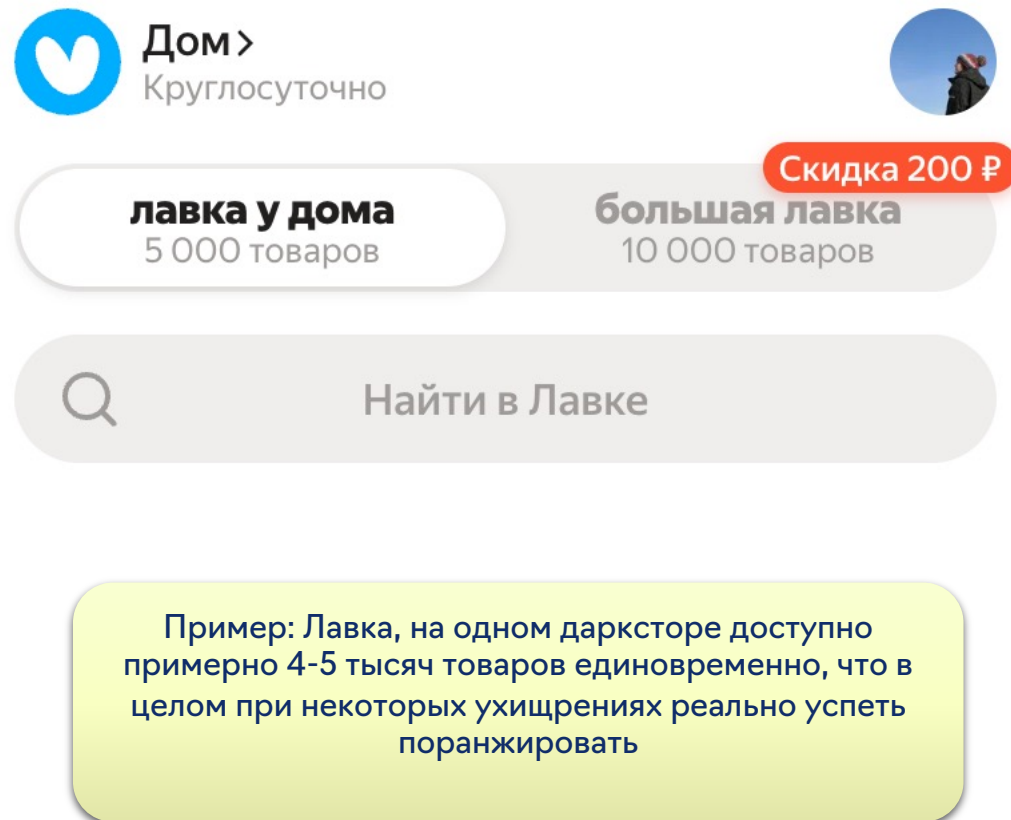
1. Допустим, мы богаты, мы можем посчитать где-то заранее эти числа для декартового произведения пользователей и айтемов (e.g. *в оффлайне*)
2. Загрузить результаты в быстрое на чтение KV-хранилище и отдавать результаты из него
3. В целом иногда даже можно так сделать, но на практике
4. Если пользователь уже положил в корзину булочку, мы знаем, что неплохо попробовать порекомендовать к ней кофе. С оффлайн-рекомендациями такое сделать не получится

Из плюсов:
Можно очень тяжелые модели

Минусы:
Рантайм-логику не учтем

Вариант 2: Одностадийная система

1. Допустим, у вас не очень много айтемов
2. Тогда вы можете успевать проскорить все прямо в рантайме
3. Тогда проблемы не существует, и этот кейс вполне себе возможен



Вариант 3: Типичная рекомендательная система

Приходим к популярной конструкции

1. Запрос прилетает в ваш рантайм
2. Быстрыми, но часто сложными эвристиками генерируем небольшое количество кандидатов из очень большого множества
3. Тяжелой моделью подсчитываем, насколько хорошо каждый кандидат подходит пользователю
4. Ранжируем
5. Кайфуем



О чем сегодня

Сегодняшняя лекция будет в большей степени освещать этап именно кандидатогенерации

В следующей лекции мы подробнее посмотрим на следующие этапы:

Ранжирование -- финальный и самый тяжелый этап, где применяются модели, которые оптимизируют финальную метрику

На последнем этапе с бизнес-логикой мы переставляем айтемы в финальной выдаче согласно некоторым продуктовым (но иногда и ML-ным соображениям)



Прerequisites

Чтобы тут рассуждать дальше, нужно понимать базовое устройство моделей этапа ранжирования

Обычно для продуктовой задачи рекомендаций это устроено так

У нас есть фронтовые логи действий пользователя в приложении / на сайте

На каждый момент времени из фронтовых логах у нас откуда-то есть рассчитанные пользовательские, айтемные и контекстные статистики

Комбинируя эти данные, мы получаем датасет, таргет в котором мы обычно так же строим из самих фронтовых логов (клик, добавление в корзину, еще что-то; вопрос негативов пока оставим за скобками)

На этом датасете мы уже и обучаем нашу мощную ранжирующую модель. Дальше говоря про позитивы и негативы, я буду иметь в виду позитивы и негативы этого датасета

У этого этапа есть свои метрики качества, о которых мы поговорим в следующей лекции

Важный момент

NB: иногда ключевых этапов бывает больше двух (retrieval и ranking), чаще всего бывает несколько последовательных этапов ранжирования, которые постепенно сужают множество кандидатов

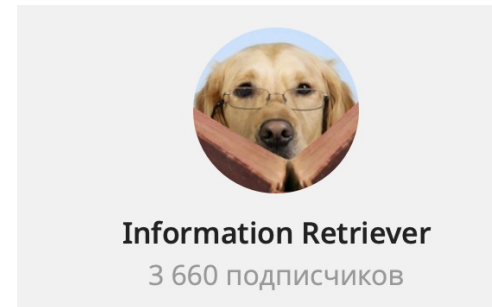


О похожести задач поиска и рекомендаций

Задача поиска и задача рекомендаций очень близки, поэтому их вместе иногда выделяют в научную область, которая называется Information Retrieval



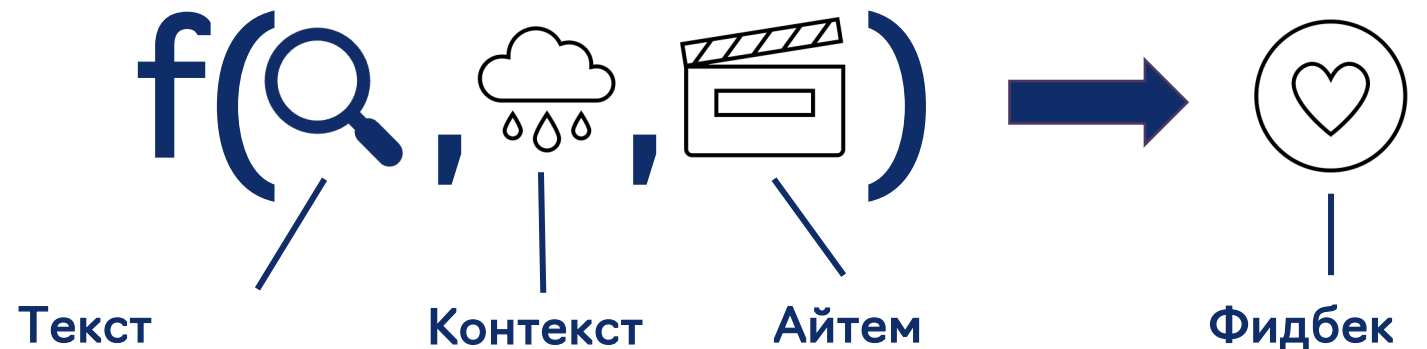
Information Retrieval



подписывайтесь на канал Кирилла

О похожести задач поиска и рекомендаций

Можно представлять, что в задачах отличается запрос: в поиске – это текст, а в рекомендациях – пользователь



О похожести задач поиска и рекомендаций

Можно представлять,
что в задачах отличается



Пользователь

Контекст

Айтем



Фидбек

На практике многие современные поисковые
системы персонализированы и запрос состоит из
текста и пользователя



Текст

Контекст

Айтем



Фидбек

О похожести задач поиска и рекомендаций

Таким образом поиск в целом устроен примерно так же, есть какое-то очень большое множество документов, которое мы сужаем до финальной выдачи в несколько этапов

Поиск – самый понятный пример системы, где этапов ранжирования может быть довольно много по вычислительным соображениям



Важное следствие

Часто бывает, что некоторые айтемы никогда не нужны в выдаче:

- Для UGC-видео – это gruesome или откровенный контент
- Для e-grocery – обычно нельзя показывать табак или алкоголь (даже если по локальным законам их можно продавать)
- Для ecom'a в широком смысле – обычно по продуктовым соображениям нельзя рекомендовать товары 18+
- Для Netflix-like – локальные ограничения по доступности или по типу лицензии

Понятно, что такие айтемы имеет смысл фильтровать на самом раннем этапе, чтобы не тратить на них дорогие вычислительные ресурсы более поздних стадий

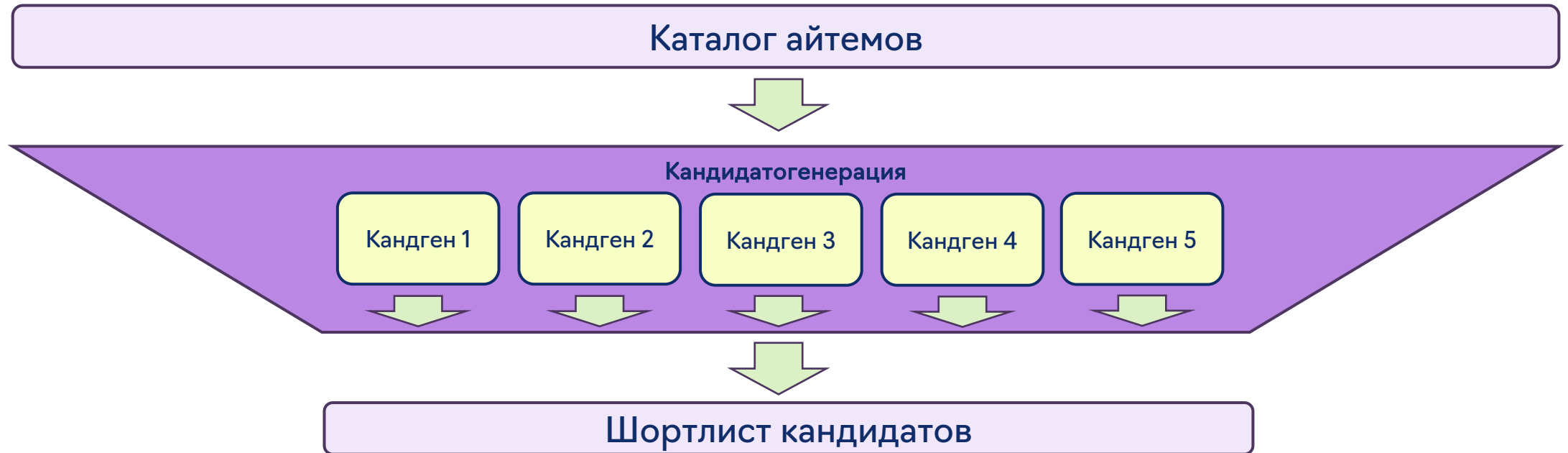
Ну и если фильтровать на последних стадиях, выдача может быть короткой, при том, что среди наших документов могут еще быть релевантные

Кандидатогенерация и ее метрики качества

Часть 2

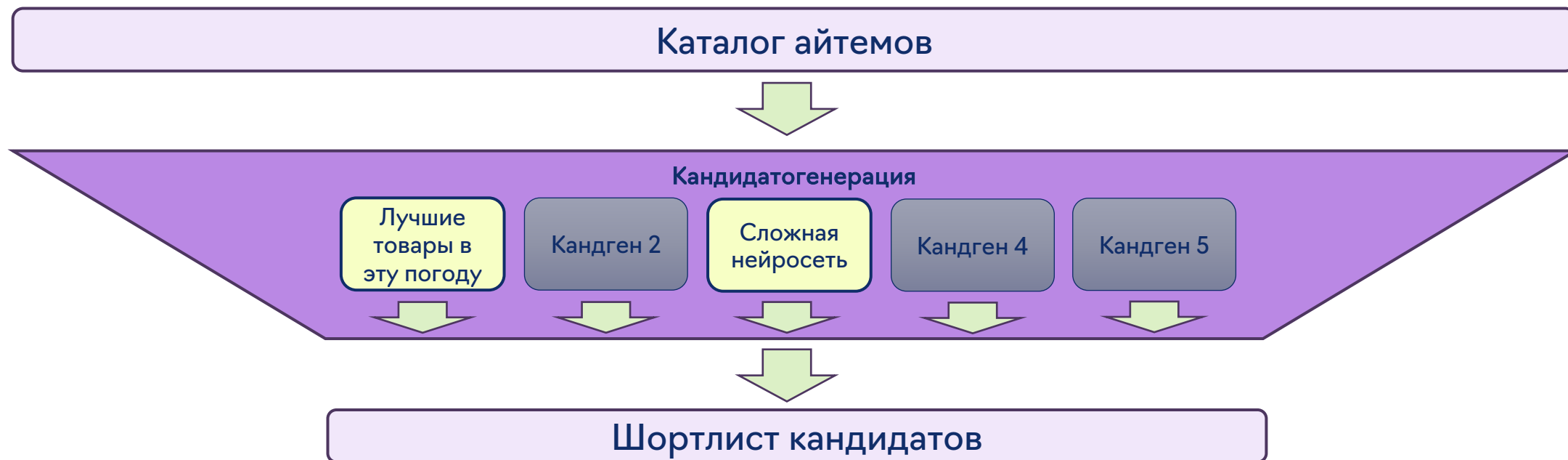


Кандидатогенераторы



Почти всегда этап retrieval'a представляет из себя композицию из нескольких разных стратегий наборов кандидатов – будем называть эти стратегии кандидатогенераторами

Кандидатогенераторы



Один кандидатогенератор может набирать, например, товары которые хорошо покупают в ту погоду, которая сейчас за окном, а другой – выдавать результаты работы сложной нейросетевой модели

Поскольку на этом этапе нужно резко снижать количество айтемов в рассмотрении, тут часто прибегают к большому количеству разных эвристик, которые по сути и соответствуют простейшим кандидатогенераторам

Что такое хороший кандген?

Пусть есть истинная полезность айтема для фиксированного пользователя:

$$u(i)$$

Если бы ограничений не было, на этапе ранжирования мы бы искали:

$$\operatorname{argmax}_{i \in \mathcal{I}} u(i)$$

Но пространство слишком большое.

Когда мы строим кандидатогенератор, мы учимся строить подмножество

$$\mathcal{C}_K \subset \mathcal{I},$$

чтобы потом на этапе ранжирования посчитать

$$\operatorname{argmax}_{i \in \mathcal{C}_K} u(i)$$

Что такое хороший кандген?

Пусть есть истинная полезность айтема для фиксированного пользователя:

Если бы ограничений не было, то при ранжировании мы бы искали

Но пространство слишком велико

Когда мы строим кандидата, мы хотим построить подмножество

чтобы потом на этапе ранжирования

$$\operatorname{argmax}_{i \in C_K} u(i)$$

Проблема. У нас нет настоящей полезности, только оценка

$$\hat{u}(i),$$

основанная на **прошлых** данных.

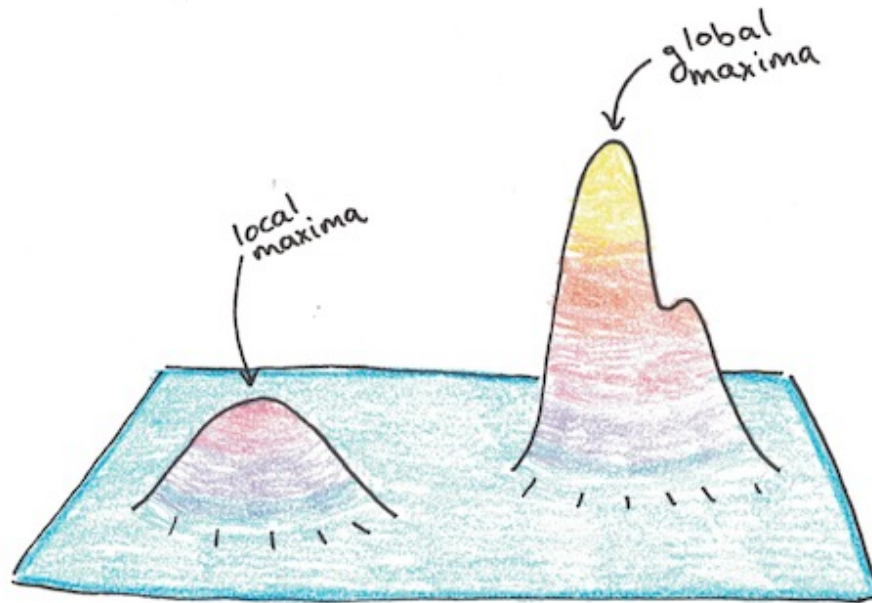
Отсюда фундаментальная проблема: оценка хорошо работает на тех примерах, где было много сигнала. И плохо – там, где данных мало.

Exploration / Exploitation Tradeoff

Если при выборе C_K мы полностью доверяем $\hat{u}(i)$, мы выбираем объекты с:

- высокой оценкой целевого действия
- высокой уверенностью этой оценки
- представленностью в датасете

Это **exploitation** нашего знания о пользователе



Но если мы хотим улучшить модель в будущем, нам нужно иногда выбирать объекты, где:

- оценка менее уверенная
- данных мало

Это **exploration** нового знания о пользователе

Exploration / Exploitation Tradeoff

В оценке качества

Поэтому будем
принципиально делить все
метрики качества на 2
группы:

- эксплуатативные
- эксплоративные

Давайте уже, наконец,
посмотрим на сами
метрики



Эксплуатативные метрики кандидатогенерации

Есть понятное соображение: наша задача в кандидатогенерации – достать как можно больше полезных айтемов, а правильный их порядок определяют более поздние и более тяжелые этапы.

Отсюда базовое желание (и самое частое, что говорят на лекциях) – растить полноту относительно наших позитивов

$$Recall@K = \frac{\# \text{ relevant items in top } K}{\# \text{ all relevant items}}$$

Дальше под полнотой мы будем в основном иметь в виду среднюю полноту по запросам.

$$\overline{Recall}@K = \frac{\sum_{requests} Recall@K}{\# \text{ requests}}$$

Отдельная ценность в том, чтобы усреднять так же по юзерно, а не только по запросу. Это бывает полезно, чтобы посмотреть, как идут дела у среднего пользователя, а не у системы в целом.

Эксплуатативные метрики кандидатогенерации

Есть еще и бинарная версия позапросной полноты –
Hitrate

$$Hitrate@K = \frac{\text{\# requests with at least 1 relevant item in top K}}{\text{\# requests}}$$

В полноте у нас по сути чуть более детализированный знаменатель, где вместо 0 или 1 под знаком суммы стоят ровно *Recall@K*.

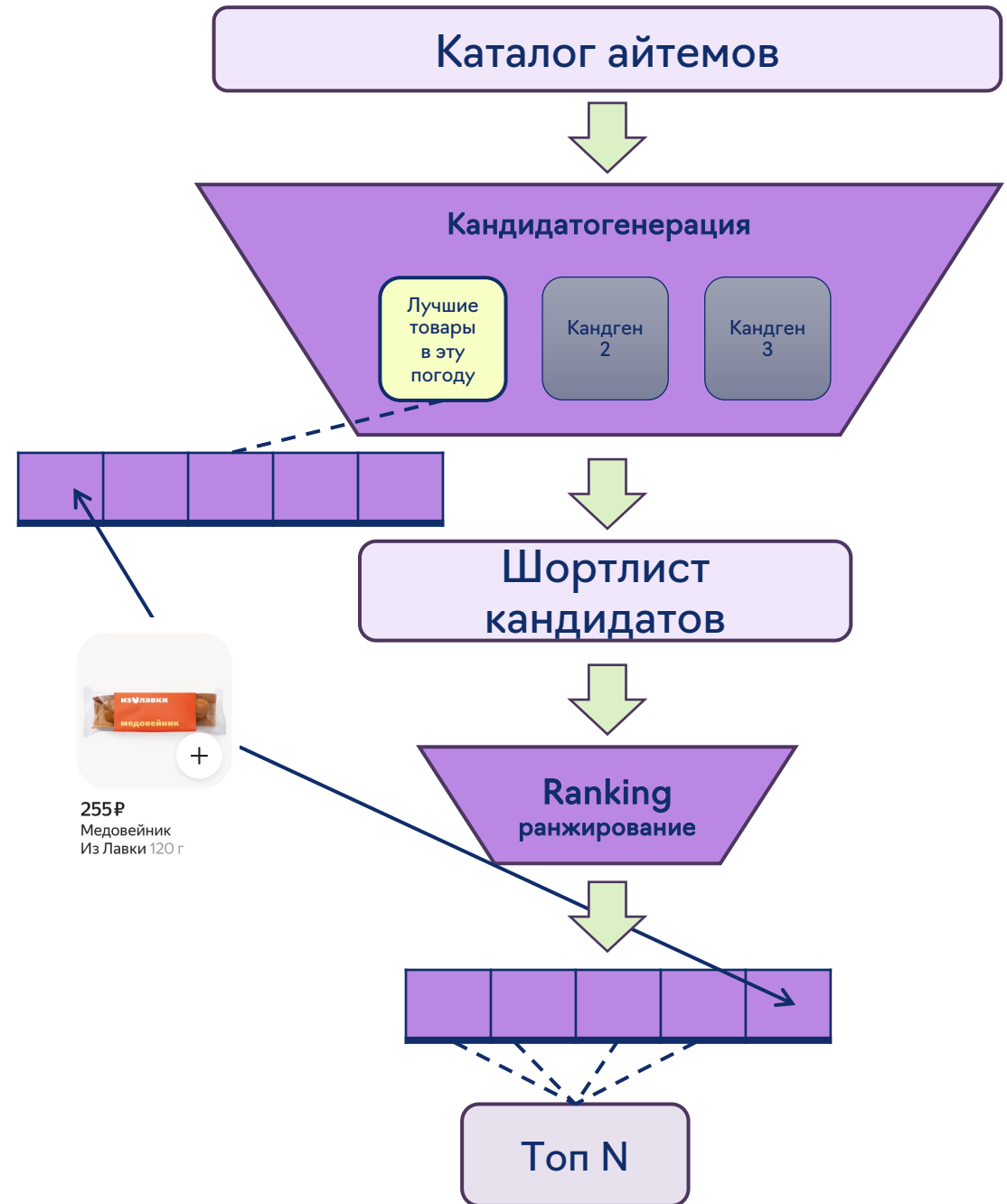
По сути обе эти метрики немного по-разному пытаются показать, в какой доле запросов у нас есть что-то релевантное в выдаче в топ K.

Проблемы:

Относительно чего измерять полноту?

Измерять полноту относительно конечного датасета может быть не очень хорошо. Этап ранжирования может быть устроен так, что он не пропустит «хороший» айтем в финальную выдачу, и измеренная нами полнота может не представлять практической ценности.

Выход: измерять полноту относительно попадания в выдачу этапа ранжирования (или более общо: следующего этапа).



Проблемы:

Относительно чего измерять полноту?

На практике это не всегда просто и существует некоторый tradeoff:

- Если полноту измерять относительно датасета, есть неточность, описанная выше
- Если полноту измерять относительно следующего этапа, мы связываем две модели и создаем сложность инженерного характера: их всегда придется переобучать вместе.

Проблемы: Выбор K

Когда мы измеряем Recall@K – мы по сути фиксируем размер выдачи кандидатогенератора, условно, уже заранее принимаем некоторое инженерное решение, относительно которого мы хотим измерить кандидатогенератор.

При этом формы кривых $\text{Recall}(K)$ могут быть существенно разными для двух кандидатогенераторов. Это может подсказать нам, сколько именно айтеров с этого кандидатогенератора стоит взять.

Но и это не совсем беспроблемная история. В реальности некоторые кандгены считаются дешевле других.

Поэтому нужно смотреть на $\text{Recall}(K)$ относительно Latency.

Эксплоративные метрики

$$\text{Novelty}(C_u) = \frac{1}{K} \sum_{i \in C_u} \left(\frac{1}{|H_u|} \sum_{j \in H_u} d(i, j) \right)$$

$$\text{Diversity}(C_u) = \frac{2}{K(K-1)} \sum_{i < j} d(i, j)$$

$$\text{Serendipity}(C_u) = \frac{1}{K} \sum_{i \in C_u} \hat{u}(i) \cdot \left(1 - \frac{1}{|H_u|} \sum_{j \in H_u} s(i, j) \right)$$

Эксплоративные метрики

$$\text{Coverage}_{item} = \frac{|\bigcup_u C_u|}{|\mathcal{I}|}$$

- Можно определять так же перцентильно
- Если значение низкое – высок риск фидбэк лупа и вырождения модели

Более высокоуровнево про tradeoff

$$Score(C_K) = \underbrace{\sum relevance(i)}_{\text{exploit}} + \lambda \cdot \underbrace{Diversity(C_K)}_{\text{explore}}$$

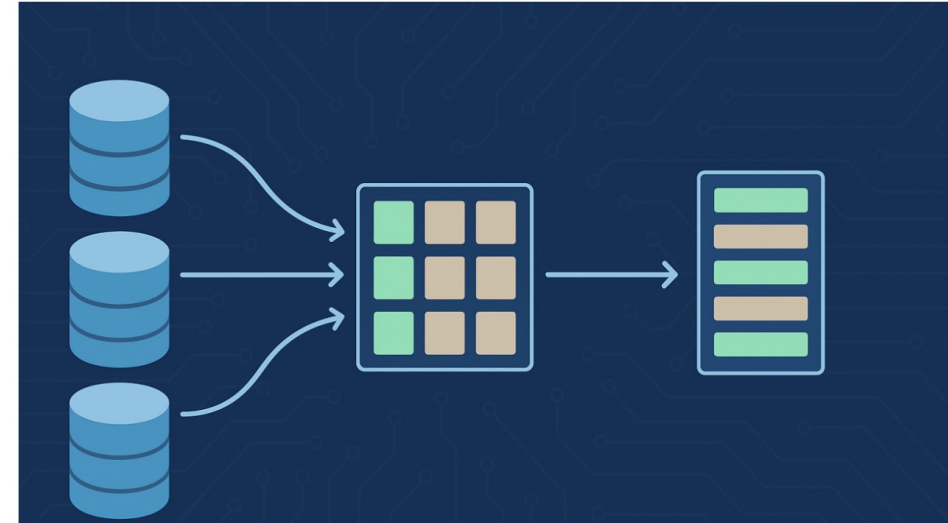
λ — это по сути философия продукта.

$\lambda = 0$ соответствует чистому эксплойту, а слишком большая -- чистому шуму

**Но это же все про один
кандген? А их много!**

Методы комбинирования кандидатогенераторов

$$\max \text{Utility}(C) \quad \text{s.t.} \quad \text{Cost}(C) \leq B$$



Optimizing Candidate Generation at Scale with Mixigen



Michael Roizner ✓

ML/AI Engineer at Meta | ex-X, ex-Microsoft, ex-Yandex



April 25, 2025

A new approach to dynamic allocation of candidate sources in recommender systems.

Про измерение в онлайн

✦ Обзор от ИИ



Сетевой эффект в A/B-тестах — это нарушение изоляции групп, при котором действия пользователей экспериментальной группы влияют на метрики контрольной группы, искажая результаты. Это типично для социальных сетей, маркетплейсов и сервисов с двусторонними рынками (например, такси, где водители ограничены), делая классическое рандомизированное тестирование неточным. [🔗](#)

Виды и примеры кандидатогенераторов

Часть 3



Статические топы

Просто список разных айтемов, по чему-то отсортированный.

В простом варианте строятся в оффлайне и грузятся на тачки настолько часто, насколько успеваете

В рантайме лежат в памяти и мгновенно используются для генерации кандидатов.



Статические топы

Например, по популярности

- клики
- Покупки
- CTR, pCTR
- wCTR, wpCTR

и свежести

- новинки
- набирающие популярность

за разные периоды времени



Кворумы

Попытка при помощи топов сделать что-то персональное, основываясь на идее TF-IDF

Идея: пользователей будем считать “документами”, а их айтемы-позитивы – “термами”

Для того, чтобы конструкция переносилась на новые айтемы вне “обучения”, введем свертку по категориям айтемов, чтобы привычный TF-IDF стал чем-то в духе

$$\sum_{category} TF(item, category)UF(user, category)IDF(category)$$

Кворумы

$$\sum_{category} TF(item, category)UF(user, category)IDF(category)$$

Определим term-frequency просто как средний набор категорий среди пользователей его употреблявших, где $UF(user, category)$ – вес категории у пользователя

$$TF(item, category) = \frac{\sum_{user \in U(item)} UF(user, category)}{|U(item)|}$$

IDF определим более-менее обычным образом

$$IDF(category) = \min \left(-\log \overline{UF(user, category)}, \gamma \right)$$

Как тогда набрать айтемы по этой штуке, когда пришел пользователь:

- Построим топы по TF по категориям в оффлайне, положим их на тачку
- На практике берут из каждого топа по N документов, считают для объединения эту функцию и берут топ

KNN-индексы

Идея, которая долгое время была (да и в основном остается) главной в кандидатогенерации.

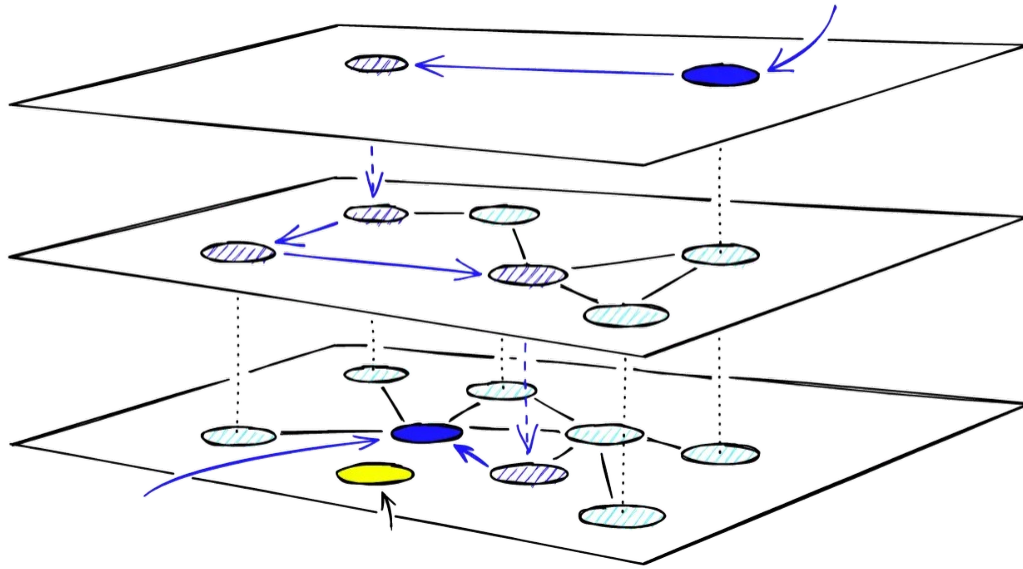
При помощи моделей, который эмбедают пользователей и айтемы в одно пространство, можно погенерировать кандидатов так:

- Построить эмбединги всех айтемов
- Построить по ним структуру данных, которая за приемлемое время умеет по пользовательскому эмбеду находить ближайшие айтемы
- Положить ее на тачки
- В рантайме откуда-то брать пользовательский эмбед и по нему извлекать айтемы из этой структуры данных

Можно медленно искать точно (KNN-индексы) или быстро приближенно (ANN-индексы).

Мы посмотрим на ANN-индекс подробнее на примере HNSW.

HNSW



Hierarchical Navigable Small World – структура данных, которая по точке в пространстве приближенно ищет N ближайших соседей из заранее заданного множества

Концептуально:

Построение

- Все точки раскладываются в иерархическую систему слоев, каждый из которых – small-world-граф: между любыми двумя точками существует короткий путь (aka 6 рукопожатий)

Поиск

- Пока не дойдем до последнего слоя, жадно ищем ближайшую к целевой на слое, а потом проваливаемся по ней вниз
- На последнем слое перебираем чуть побольше для пущей точности

HNSW

Преппроцессинг

Берем наше множество точек и для каждой точки по очереди

- Определяем уровни, на которые она попадет
- Максимальный уровень сэмпим из

$P(\text{maxLevel} = k) \sim \text{Geometric}(e^{-\lambda k}, \text{levelCount})$ или

$P(\text{maxLevel} = k) = e^{-\lambda k} (1 - e^{-\lambda k})^{\text{levelCount}}$, где

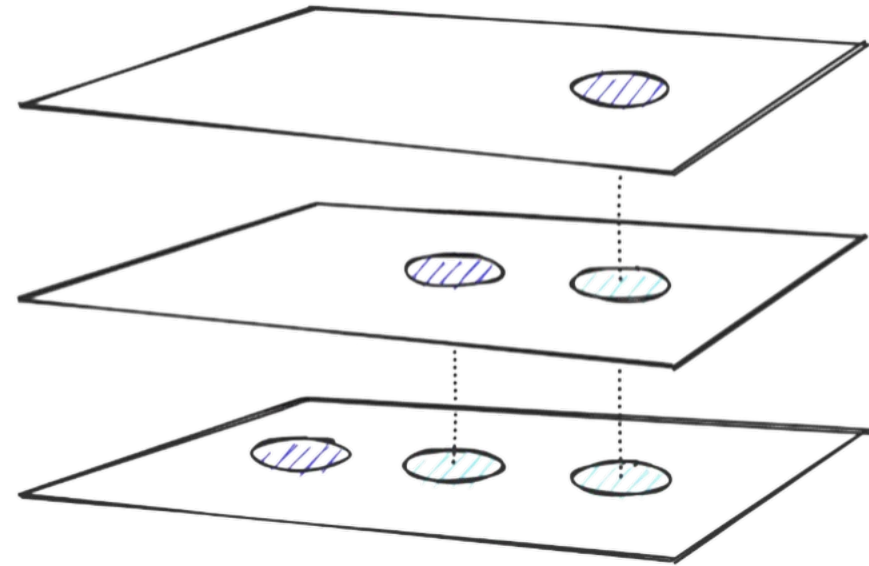
$$\lambda = \frac{1}{M}$$

- Во все меньше него попадает автоматически.
По сути просто какое-то красивое экспоненциальное замедление
- Здесь M – гиперпараметр, отвечающий за максимальную степень вершины в графе (увидим дальше)

HNSW

Препроцессинг

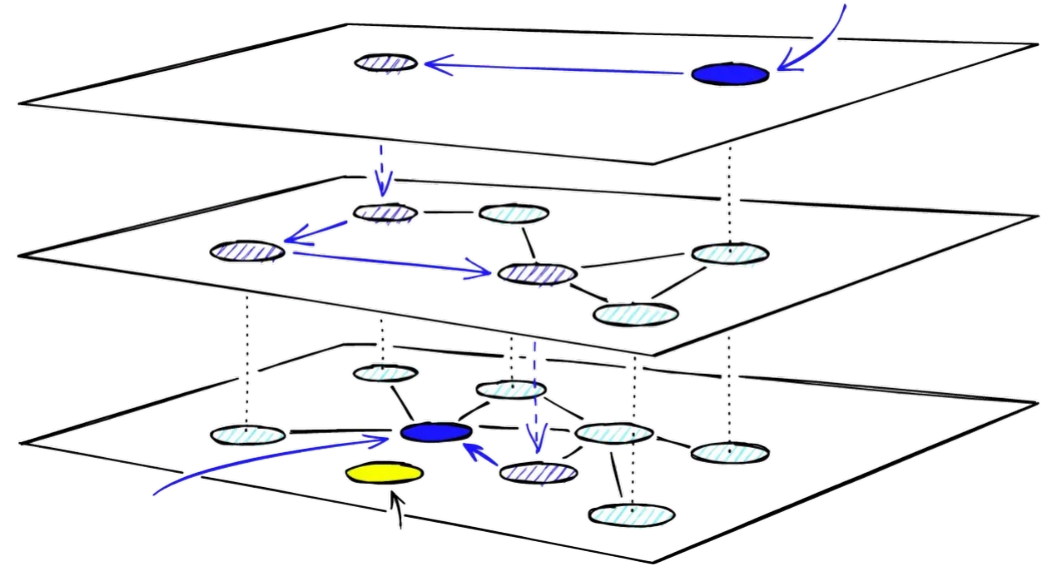
- Пока не дойдем до первого слоя, где есть точка, поддерживаем на слое жадного ближайшего соседа: на самом высоком слое берем честно посчитанного ближайшего, спускаемся на слой ниже по этой же вершине (она там есть по построению) и идем по ребрам, пока расстояние уменьшается (всегда выбираем жадно)



- На остальных слоях своего рода BFS-ом ищем M ближайших: у BFS'а есть максимальный размер очереди – `efConstruction`. При этом в очередь добавляются новые вершины только если они жадно уменьшают расстояние до новой точки. Соединяем эти M точек с нашей ребрами

HNSW
Поиск

- Стартуем с самого высокого слоя, на нем очень мало точек и можно просто руками найти ближайшую
- На каждом следующем слое идем от нее по вершинам-соседям, пока можем жадно уменьшить расстояние до искомой точки



- На самом последнем слое снова делаем странный BFS: ограничиваем размер очереди до `efSearch`. При этом в очередь добавляем новые вершины только если они жадно уменьшает расстояние до новой точки.
- По дороге поддерживаем `heap` с топ-N соседями, ими и отвечаем.

Важный исторический пример: ALS

Одна из первых продуктивных идей в рекомендациях:
советовать вам то, что понравилось людям, которым
понравилось то, что понравилось вам

Это еще называют коллаборативной фильтрацией

- Можно попробовать сделать как-то наивно, но популярнее делать иначе
- Построим матрицу взаимодействий и что-нибудь с ней сделаем
- Например, получим эмбедаы айтемов и пользователей, как в ALS
- Или прямо скоры, как в SLIM (на них сегодня смотреть не будем)

		<u>айтемы</u> j					
i пользователи	3	1	4	1	5	9	
	2	6	5	3	5	8	
	9	7	9	3	2	3	
	8	4	6	2	6	4	
	3	3	8	3	2	7	
	9	5	0	2	8	8	

например, оценка
пользователем i айтема j

Важный исторический пример: ALS

Он же – Alternating Least Squares

Идейно:

- Хотим приблизить исходную матрицу в виде произведения двух

$$R \approx \hat{R} = U \times V^T$$

- В одной пользовательские эмбеда, в другой – айтемные
- Тогда для известных оценок можно выписать вот такой лоссик (сразу с bias'ами)

$$L = \sum_{(u,i) \in R} (\hat{r}_{ui} - r_{ui} - b_u - b_i)^2 + \lambda_{emb} \left(\sum_{u \in U} \|p_u\|^2 + \sum_{i \in I} \|q_i\|^2 \right) + \lambda_{bias} \left(\sum_{u \in U} b_u^2 + \sum_{i \in I} b_i^2 \right)$$

Можно поочередно “фризить” пользовательские и айтемные параметры и решать систему уравнений, которая получается при взятии производной

Это метод наименьших квадратов, отсюда название

Важный исторический пример: ALS

То, что мы описали на предыдущем слайде – eALS, он же explicit ALS

- Поскольку лоссик считается по явным оценкам
- Что делать, если у нас неявный фидбек?

Вообще вариантов масса, но популярный подход – iALS, он же implicit ALS

- Все как с explicit'ом, но
- Лосс считается по всей матрице, предполагая неизвестные интеракции нулями

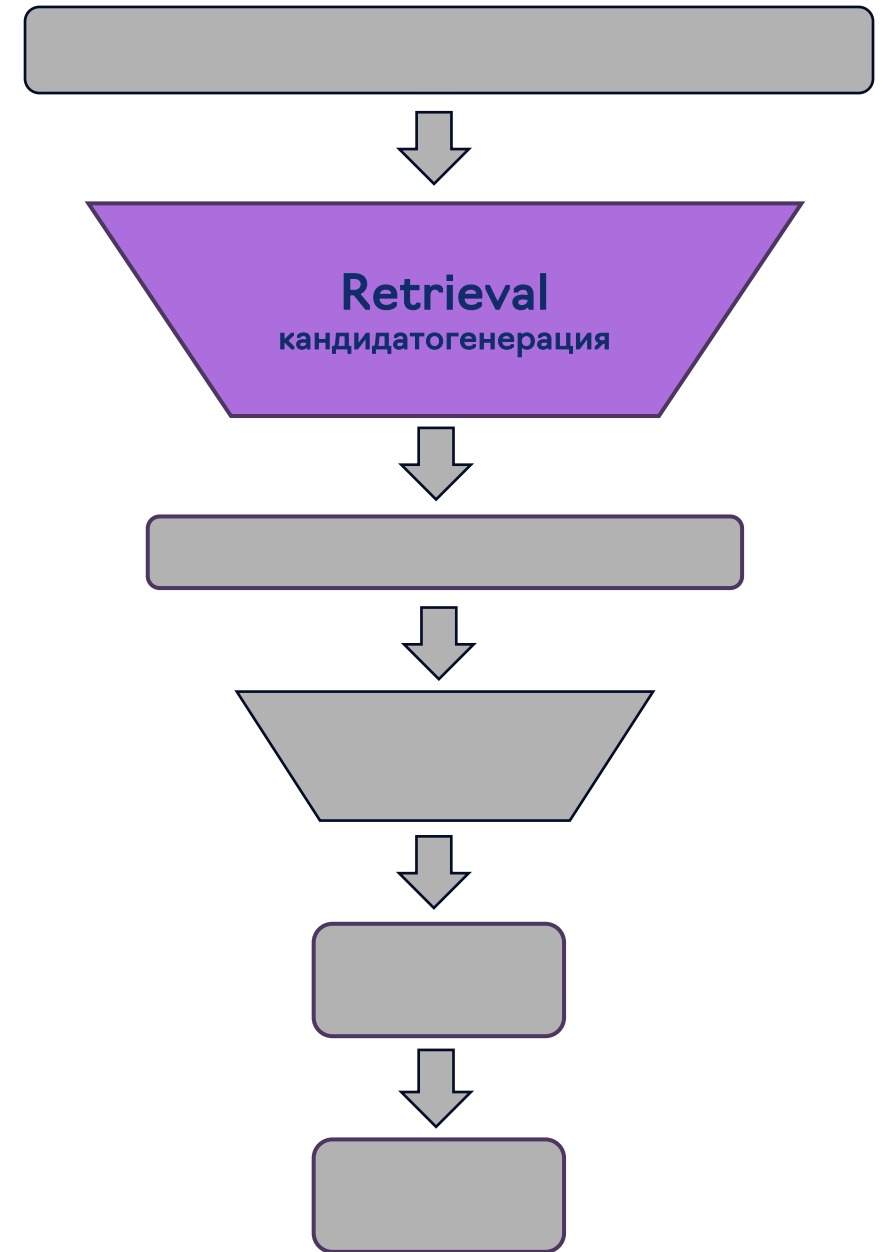
$$L = \sum_{i,j} C_{ij} (R_{ij} - U_i V_j^T)^2 + \lambda \dots$$

Добавляется “уверенность” в каждой оценке тем более большая, чем сама оценка

$$C_{ij} = 1 + \alpha R_{ij}$$

Использование ALS в кандидатогенерации

- Считаем пользовательские и айтемные эмбеды
- Пользовательские эмбеды кладем в какой-нибудь key-value storage
- В рантайме забираем из него эмбед пользователя
- На точки раскладываем ANN-индекс
- Ищем ближайшие к пользовательскому эмбеду в ANN-индексе по такой близости, чтобы оптимизировала максимум (легко построить)



Альтернативы ANN: MoL

$$\text{score}_{MoL}(x, u) = \frac{1}{\tau} \sum_{i=1}^{K_u} \sum_{j=1}^{K_x} \pi_{i,j}(x, u) \langle \mathbf{e}_u^i, \mathbf{e}_x^j \rangle,$$

Общая базовая идея: у нас есть GPU, они отлично умеют считать много косинусов одновременно, давайте сложим все на карточку, т.к. мы богаты

Конкретно MoL: построим много разных пользовательских и документных эмбедов и насчитаем над скалярными произведениями что-то типа софтмакса, должно получиться лучше

Альтернативы ANN: MoL

$$\text{score}_{MoL}(x, u) = \frac{1}{\tau} \sum_{i=1}^{K_u} \sum_{j=1}^{K_x} \pi_{i,j}(x, u) \langle \mathbf{e}_u^i, \mathbf{e}_x^j \rangle,$$

Общая базовая идея
считать много косинусов
все на карточку, т.е.

Конкретно MoL: по
и документных эмбедингов
произведениями ч
получиться получи

Чтобы успевать посчитать все, что нужно,
воспользуемся идеей h-indexer:

- Посэмплируем несколько процентов от всех документов
- Честно посчитаем скор для пользователя и этих документов
- Возьмем скор k-ого, это будет трешолд для фильтрации при полном сканировании
- Из-за софтмаксности мы сильно срежем количество вычислений

I2i списки

Обучим очень крутую i2i-модель:

- возьмем все контентное, что сможем
- поднимем крауд на похожесть
- обучим под его скоры модель

Теперь к каждому айтему можно:

- построить топчик по похожести (которая чисто контентная)
- переранжировать неперсональной контентной моделью целевого действия

В рантайме будем брать позитивы из истории, выбирать из них разнообразные (про алгоритмы разнообразия расскажут в следующей лекции) и по ним лукапить топы списков для этапа ранжирования.

Если айтемов слишком много, можно оставить только некоторые (самые репрезентативные).

Это в пруде у Дзена и в целом при некоторой фантазии (если центроиды становятся обучаемыми) эта идея развивается до семантических айдишников (будет дальше в курсе)

Что мы узнали сегодня

- Много всего про кандидатогенерацию
- Про exploration / exploitation tradeoff и feedback loop
- Про много видов разных кандгенов
- И про то как померить качество всего этого



Организационная часть про контекст и домашнее задание

Часть 4



Спасибо!

Даня Ткаченко,
Руководитель службы ML-сервисов Яндекс Лавки

Белград 2026

